

Advanced Data Management

Lab 3 — Indexing: measuring the payoff and the cost

Cycle Ingénieur · S3

Fall 2026

Goals

Measure what an index actually buys you, learn to read which index a plan uses, discover the **selectivity** point where the planner abandons the index, and quantify the write cost of indexing. Uses `adm_lab`.

Note. For clearer timings, grow `orders` first (larger tables make index effects obvious):

```
INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1+floor(random()*10000)::int, 1+floor(random()*200)::int,
       round((random()*100+5)::numeric,2),
       date '2025-01-01' + (floor(random()*365)::int)
FROM generate_series(1,450000);          -- now ~500k orders
ANALYZE;
```

1. Baseline (no index)

```
\timing on
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM customers WHERE name = '
    Customer 7777';
EXPLAIN (ANALYZE, BUFFERS)
    SELECT * FROM orders WHERE order_date BETWEEN '2025-06-01' AND '
        2025-06-10';
```

Record the plan type, time, and buffers for each.

2. Add indexes and compare

```
CREATE INDEX idx_cust_name ON customers(name);
CREATE INDEX idx_orders_date ON orders(order_date);
-- re-run the two queries above and compare
```

3. Multi-column index & the prefix rule

```
CREATE INDEX idx_oc ON orders(cust_id, order_date);
EXPLAIN SELECT * FROM orders WHERE cust_id = 42;
EXPLAIN SELECT * FROM orders WHERE cust_id = 42 AND order_date > '
    2025-06-01';
EXPLAIN SELECT * FROM orders WHERE order_date > '2025-06-01';
```

Note which of the three uses `idx_oc` and which does not.

4. Find the selectivity flip

```
CREATE INDEX idx_amount ON orders(amount);
-- vary the threshold and watch Index Scan vs Seq Scan
EXPLAIN SELECT * FROM orders WHERE amount > 104;
EXPLAIN SELECT * FROM orders WHERE amount > 60;
EXPLAIN SELECT * FROM orders WHERE amount > 20;
```

5. Expression & partial indexes

```
CREATE INDEX idx_lower_name ON customers(lower(name));
EXPLAIN SELECT * FROM customers WHERE lower(name) = 'customer 7777';
```

6. The write cost of indexing

```
-- time a bulk insert WITH the indexes present
\timing on
INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1+floor(random()*10000)::int, 1+floor(random()*200)::int,
       round((random()*100+5)::numeric,2),
       date '2025-01-01' + (floor(random()*365)::int)
FROM generate_series(1,100000);

-- now DROP the indexes and repeat to compare
-- DROP INDEX idx_orders_date, idx_oc, idx_amount;
```

7. Exercises to submit

1. For the point lookup, report the seq-scan time vs. the index-scan time and compute the speedup factor.
2. Demonstrate the leftmost-prefix rule with the three queries from Part 3; state which used `idx_oc`.
3. By varying the `amount` threshold, find the approximate fraction of rows at which the planner switches from Index Scan to Seq Scan. Report that selectivity.
4. Show that a plain index on `name` does *not* serve `lower(name)=...`, then that the expression index does.
5. Measure the insert time for 100k rows with the indexes present vs. after dropping them. Report the slowdown.
6. Short answer: name one column you would index in `adm_lab` and one you would not, with one line of reasoning each.

Deliverable

A Jupyter notebook `adm_lab3_<yourname>.ipynb` with the queries, plans, and timings; the selectivity-flip finding; the write-cost comparison; and your answers to the six exercises. Attach the `.sql` you used.

Troubleshooting

- Plan still shows Seq Scan after creating an index: run `ANALYZE`; so the planner has fresh statistics, and check the query returns few enough rows.
- Timings jump around: run each query twice and use the second (warm-cache) time, but say so.